

IRON MAN AI — Manuel complet (Kali)

Audit web · WiFi · Android · Périphériques · CodeScan

1.CodeScan — Analyse statique de code

2.IRON MAN AI — Audit web (Kali)

3.IRON MAN AI — Pentest WiFi

4.IRON MAN AI — Analyse Android (APK)

5.IRON MAN AI — Audit de périphérique

6.IRON MAN AI — Contrôler votre téléphone

Généré automatiquement — cadre d'utilisation autorisé uniquement (--authorized).

CodeScan — Analyse statique de code

Manuel d'utilisation — CodeScan

Analyseur statique de code **sans API d'IA**, basé sur des règles AST et regex
et sur la base de vulnérabilités OSV.dev.

Sommaire

1. [À propos](#1--à-propos)
1. [Prérequis et installation](#2--prérequis-et-installation)
1. [Démarrage rapide](#3--démarrage-rapide)
1. [Référence des options CLI](#4--référence-des-options-cli)
1. [Exemples d'utilisation détaillés](#5--exemples-dutilisation-détaillés)
1. [Fonctionnement interne (pipeline)](#6--fonctionnement-interne-pipeline)
1. [Les rapports de sortie](#7--les-rapports-de-sortie)
1. [Codes de sortie](#8--codes-de-sortie)
1. [Référence des règles](#9--référence-des-règles)
1. [Personnalisation des règles](#10--personnalisation-des-règles)

1. [Exécution des tests](#11--exécution-des-tests)
 1. [Dépannage et FAQ](#12--dépannage-et-faq)
 1. [Intégration CI/CD](#13--intégration-cicd)
 1. [Limites connues](#14--limites-connues)
 1. [Mode IRON MAN AI — audit Kali (sites web)](#15-mode-iron-man-ai-audit-kali-sites-web)
-

1. À propos

CodeScan parcourt un projet (dossier local ou dépôt GitHub cloné) et repère :

- les **failles de sécurité** classiques : injection SQL, XSS, exécution de code/commandes, désérialisation non sûre, `except: nu...` ;
- les **secrets exposés** : clés AWS/Google/Stripe, tokens GitHub/Slack/npm, clés privées, mots de passe en clair — par patterns regex **et** par calcul d'**entropie de Shannon** ;
- les **failles de configuration** : CORS permissif, regex dynamique utilisateur (ReDoS), superglobales PHP lues directement, `SELECT *` ;
- la **dette technique et qualité** : TODO/FIXME, fonctions trop longues/complexes (cyclomatiques **et cognitives**), imbrication excessive, trop de paramètres, `==` au lieu de `===`, blocs vides, Nombres magiques, code commenté, `console.log` de débogage, copie profonde via JSON ;
- la **performance/asynchrone (JS)** : I/O synchrones bloquantes dans des gestionnaires, I/O successives en boucle (N+1), boucles quadratiques $O(n^2)$, copie profonde coûteuse ;
- les **dépendances vulnérables** : croisement de `requirements.txt`, `package.json`, `composer.json` ... avec la base de CVE **OSV.dev** ;
- une **note de qualité /100 + lettre (A→F)** qui synthétise tout le projet.

Contrairement à un assistant IA, CodeScan ne « devine » rien : chaque résultat provient d'une règle déterministe (AST Python ou regex) et est associé à une sévérité, une description et une recommandation.

2. Prérequis et installation

Prérequis	Rôle
Python ≥ 3.10	Interpréteur (le cœur utilise uniquement la bibliothèque standard)
git (optionnel)	Nécessaire uniquement pour le mode <code>--repo</code> (clonage)
requests (optionnel)	Nécessaire uniquement pour la vérification OSV.dev

```
# Depuis le dossier codescan/  
pip install -r requirements.txt      # installe requests (facultatif)
```

Astuce Windows : si `python` n'est pas reconnu, utilisez le lanceur

`py` à la place (ex. `py main.py --path ./mon_projet`).

Vérifier l'installation :

```
py main.py --version      # affiche la version  
py main.py --help        # affiche l'aide complète
```

3. Démarrage rapide

```
# 1. Analyser un dossier local (résumé console uniquement)  
py main.py --path ./mon_projet  
  
# 2. Analyser un dossier et générer un rapport HTML  
py main.py --path ./mon_projet --output report.html  
  
# 3. Analyser un dépôt GitHub (cloné puis supprimé automatiquement)  
py main.py --repo https://github.com/user/repo --output report.json  
  
# 4. Ne garder que les failles haute/critique  
py main.py --path ./mon_projet --output report.html --severity-min=high  
  
# 5. Démo sur le projet d'exemple fourni  
py main.py --path examples --output report.html --verbose  
  
# 6. Rapport « épuré » : sans qualité de code ni note  
py main.py --path ./mon_projet --no-quality --no-score --output report.html
```

Le résultat s'affiche immédiatement dans le terminal ; le rapport (HTML ou JSON) est écrit dans le fichier indiqué par `--output`.

4. Référence des options CLI

```
usage: CodeScan [-h] [--path CHEMIN] [--repo URL] [-o FICHIER]
               [--severity-min {critical,high,medium,low}]
               [--no-deps] [--no-quality] [--no-score]
               [--rules FICHIER] [-v] [--version]
```

Option	Description	Défaut
<code>--path</code> <code><CHEMIN></code>	Dossier local du projet à analyser.	—
<code>--repo</code> <code><URL></code>	URL GitHub à cloner puis analyser (<code>https://github.com/user/repo</code>).	—
<code>-o, --output</code> <code><FICHIER></code>	Rapport de sortie. Extension <code>.json</code> ou <code>.html</code> (le format est déduit de l'extension). Sans cette option, résumé console seul.	—
<code>--severity-min</code> <code><NIVEAU></code>	Sévérité minimale des résultats retenus. Les niveaux inférieurs sont filtrés du rapport.	<code>low</code>
<code>--no-deps</code>	Désactive l'interrogation réseau d'OSV.dev (analyse hors ligne).	<code>false</code>
<code>--no-quality</code>	Désactive l'analyseur de qualité de code (métriques de fonction, lignes longues, async/perf JS, égalité faible...). Les règles AST Python et regex de sécurité restent actives.	<code>false</code>
<code>--no-score</code>	Ne calcule ni n'affiche la note /100 ; la clé <code>score</code> disparaît des rapports JSON/HTML.	<code>false</code>
<code>--rules</code> <code><FICHIER></code>	Chemin d'une base de patterns alternative au format JSON (voir §10).	<code>rules/patterns.json</code>
<code>--exclude</code> <code><MOTIF></code>	Exclut les fichiers dont le chemin relatif contient le motif (option répétable ou motifs séparés par des virgules ; jokers <code>*</code> / <code>?</code> acceptés). Ex. <code>--exclude tests,migrations</code> ou <code>--exclude '*.min.js'</code> .	—
<code>-v, --verbose</code>	Affiche le détail : fichiers explorés, messages des analyseurs, liste complète des findings.	<code>false</code>
<code>--version</code>	Affiche la version et quitte.	—
<code>-h, --help</code>	Affiche l'aide et quitte.	—

Contraintes d'usage :

- `--path` et `--repo` sont **mutuellement exclusifs** mais l'un des deux est **obligatoire** ; sinon CodeScan renvoie une erreur (code 2).
- `--output` doit se terminer par `.json` ou `.html` ; toute autre extension produit une erreur (code 2).

5. Exemples d'utilisation détaillés

5.1 Analyse locale avec rapport HTML

```
py main.py --path ./mon_projet --output rapport.html --severity-min=medium --verbose
```

- Analyse tous les fichiers du projet (hors dossiers ignorés et binaires).
- Écrit `rapport.html` (autonome, consultable hors ligne).
- `--verbose` liste chaque fichier exploré dans la console.

5.2 Analyse d'un dépôt GitHub

```
py main.py --repo https://github.com/django/django --output scan.json --no-deps
```

- Clone le dépôt en profondeur 1 dans un dossier temporaire.
- Analyse puis **supprime** automatiquement le clone (rien ne reste sur disque).
- `--no-deps` évite ici la (longue) requête OSV.dev sur les centaines de dépendances d'un gros projet.

5.3 Filtrage par sévérité (pour un tri rapide)

```
py main.py --path ./mon_projet --output urgent.html --severity-min=critical
```

Ne conserve que les failles **critiques**. Utile pour le tri des incidents.

5.4 Mode hors ligne

```
py main.py --path ./mon_projet --no-deps
```

Ne fait **aucun** accès réseau : les CVE de dépendances ne sont pas vérifiées. Le reste de l'analyse est inchangé.

5.5 Base de règles personnalisée

```
py main.py --path ./mon_projet --rules ./ma-base.json
```

Charge les patterns d'un fichier JSON alternatif (voir §10).

5.6 Exclure des fichiers/dossiers de l'analyse

```
py main.py --path ./mon_projet --exclude tests,migrations --exclude '*.min.js'
```

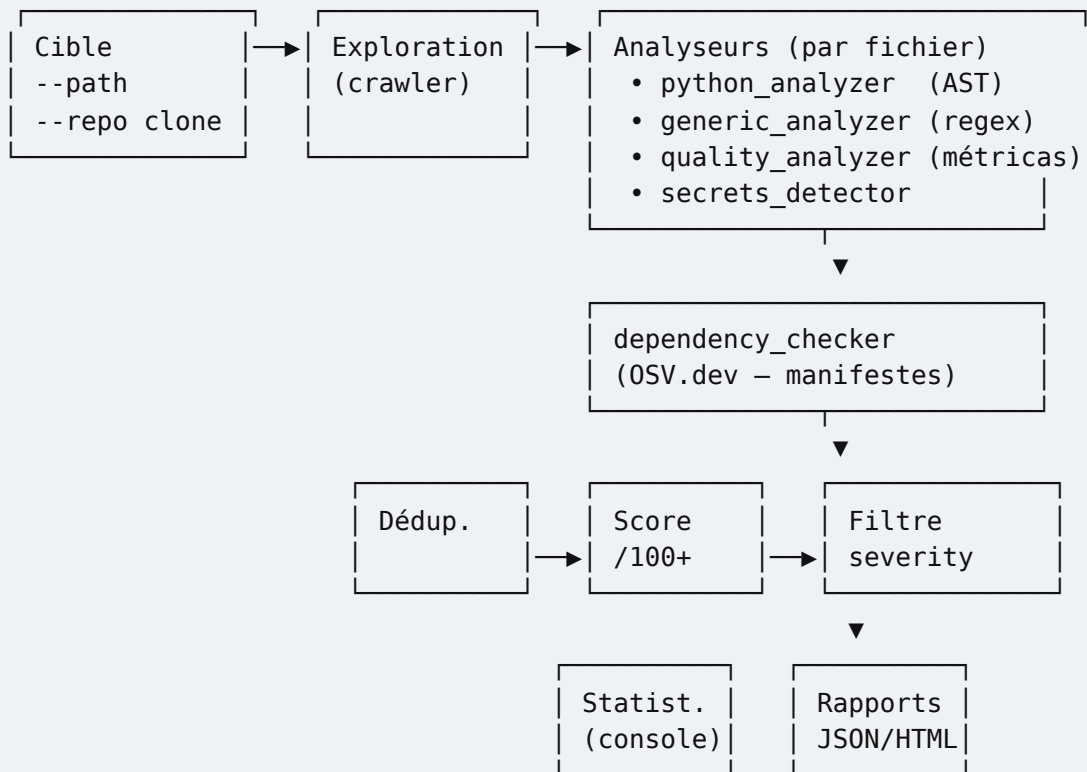
Exclut tout fichier dont le chemin relatif contient `tests` ou `migrations`, ainsi que les fichiers `*.min.js` (jokers acceptés). Pratique pour ignorer du code généré, des fixtures de test ou des bundles.

5.7 Rapport « sécurité uniquement »

```
py main.py --path ./mon_projet --no-quality --no-score --output scan.json
```

Désactive l'analyse de qualité (métriques de fonction, perf JS...) **et** le calcul de la note : seuls les findings de sécurité/dépendances/secrets restent, et le rapport JSON n'a pas de clé `score`. Utile si vous ne voulez pas de bruit qualité ou si vous préférez un score calculé par un autre outil.

6. Fonctionnement interne (pipeline)



Étapes détaillées :

1. **Récupération de la cible** — dossier local ou clonage `git clone --depth 1`.
1. **Exploration** (`scanner/crawler.py`) — parcours récursif ; ignore les dossiers `node_modules/`, `.git/`, `venv/`, `__pycache__/`, `dist/` ..., les extensions binaires, et les fichiers > 2 Mo. Chaque fichier est classé : `python`, `source`, `config`, `manifest`.
1. **Analyse Python** (`scanner/python_analyzer.py`) — parsing AST : `eval`, `exec`, `pickle/marshal`, `except: nu`, `subprocess(shell=True)`, injection SQL, `assert` de sécurité, imports dangereux, secrets nommés, et complexité de code : longueur de fonction, cyclomatique **et cognitive**, imbrication profonde, trop de paramètres, `else` après `return`, comparaison booléenne « redondante », blocs `except` vides.
1. **Analyse générique** (`scanner/generic_analyzer.py`) — regex sur le contenu : SQL, XSS, commandes, TODO/FIXME, CORS permissif, regex dynamique (ReDoS), superglobales PHP, `SELECT *` ...
1. **Analyse de qualité** (`scanner/quality_analyzer.py`, désactivée par `--no-quality`) — métriques de fonctions sur les langages à accolades (JS/TS/PHP/Java/C#/Go/...) par équilibrage d'accolades sur un code dont les chaînes/templates/commentaires sont masqués ; règles lignes : lignes > 120, fichiers > 500 lignes, code commenté, nombres magiques, `==` vs `===` ; règles de débogage JS (`console.log`, `alert`, copie profonde JSON) ; règles **performance/asynchrone** : I/O synchrones bloquantes (hors fonctions `config/init/load` et top-level), I/O successives dans les boucles

(N+1, hors `Promise.all` / `for await`), boucles $O(n^2)$ avec `.includes` / `indexOf` (hors caches `Map` / `Set`).

1. **Détection de secrets** (`scanner/secrets_detector.py`) — patterns forts, **entropie de Shannon** (seuil 4,5 bits/caractère), mots de passe en clair dans les fichiers de configuration (test appliqué à la clé seule, pas à la ligne entière). Les fichiers de verrouillage générés automatiquement (`package-lock.json` , `yarn.lock` , `pnpm-lock.yaml` ...) sont ignorés.
1. **Vérification des dépendances** (`scanner/dependency_checker.py`) — parsing des manifestes puis requêtes **batch** vers `api.osv.dev` .
1. **Post-traitement — déduplication**, calcul du **score /100 + grade** sur l'ensemble déduplicé (indépendant de `--severity-min`), filtrage de l'affichage, tri, statistiques, rapport.

7. Les rapports de sortie

7.1 Résumé console

Affiché systématiquement :

```
CodeScan – Résumé de l'analyse
```

```
Cible           : .\mon_projet
Fichiers analysés : 42
Score de qualité  : 61/100 (C)
Total findings   : 12
```

Par sévérité :

```
critical 2  █
high     5  ███
medium   3  ███
low      2  █
```

Par catégorie :

```
injection      4
secrets        3
...
```

Les couleurs ANSI sont activées automatiquement si le système est un TTY

(désactivées si la sortie est redirigée, pour un pipeline). La ligne

Score de qualité n'apparaît que si le score est calculé (par défaut) ;

avec `--no-score` elle devient `Score de qualité : désactivé (--no-score)`.

7.2 Rapport JSON

Structure exacte (compatible avec des outils tiers ou du CI) :

```
{
  "meta": {
    "tool": "CodeScan",
    "version": "1.1.0",
    "timestamp": "2026-08-04T14:05:00",
    "target": "D:\\projet",
    "severity_min": "low"
  },
  "target": "D:\\projet",
  "summary": {
    "total_findings": 12,
    "files_scanned": 42,
    "by_severity": { "critical": 2, "high": 5, "medium": 3, "low": 2 },
    "by_category": { "injection": 4, "secrets": 3 }
  },
  "score": {
    "score": 61,
    "grade": "C",
    "total_findings": 12,
    "files_scanned": 42,
    "security": 6,
    "quality": 5,
    "performance": 1,
    "by_level": { "CRITIQUE": 2, "À REVOIR": 8, "MINEUR": 2 },
    "by_domain_pct": { "security": 50.0, "quality": 41.7, "performance": 8.3 },
    "weights": { "critical": 5, "high": 2, "medium": 1, "low": 0.5 }
  },
  "findings": [
    {
      "file": "app.py",
      "line": 30,
      "column": 0,
      "rule_id": "py-subprocess-shell",
      "category": "injection",
      "severity": "high",
      "title": "subprocess avec shell=True",
      "description": "subprocess avec shell=True invoque le shell système...",
      "recommendation": "Utiliser subprocess sans shell (liste d'arguments)...",
      "snippet": "subprocess.run(cmd, shell=True)",
      "language": "python",
      "cve": "",
      "source": "python_analyzer"
    }
  ]
}
```

7.3 Rapport HTML (structure Herald)

- **Autonome** : CSS embarqué, aucune ressource externe, consultable hors ligne.
- **Adaptatif** : thème sombre/clair selon les préférences du navigateur.
- **Imprimable en PDF** : `@media print` format **A4** (`@page { size: A4; margin: 12mm }`), cartes et relevés avec `break-inside: avoid` , `print-color-adjust: exact` — « Imprimer en PDF » dans le navigateur donne un document fidèle sans coupes.
- **Structure** (parité avec le rapport de référence Herald) : 1. **en-tête** : titre, badge « **Sans IA** », cible, version, date ; 2. **hero score** : grand chiffre `score/100` (ex. `61/100`), **lettre de grade colorée** (A→F) et barre de progression de la note ; 3. **7 cartes de synthèse** : Relevés, Critiques, À revoir, Fichiers, Sécurité, Qualité, Performance ; 4. **répartition par catégorie** avec barres proportionnelles ; 5. **findings groupés par niveau lisible** : `CRITIQUE` → `À REVOIR` → `MINEUR` (les 3 sections apparaissent toujours, même vides — parité Herald), chaque entrée avec **badge rule_id**, titre, badge de sévérité (`critical` rouge foncé, `high` orange, `medium` jaune, `low` bleu), **fichier:ligne**, description, **recommandation** et extrait de code en `<pre>` échappé.

Sans score (`--no-score`), le hero et les cartes disparaissent, mais le détail par niveau reste.

7.4 Score de qualité /100 et grade

Le score synthétise l'état global d'un projet en **une note /100** et une lettre **A → F**. Il est calculé par `scanner/scorer.py` :

```
WEIGHTS      = {critical: 5, high: 2, medium: 1, low: 0.5}
densité      = Σ(poids[sev] × nb[sev]) / fichiers
crit_ratio   = nb(critical) / fichiers
raw          = 100 / (1 + densité/15.5) × (1 - 0.5 × crit_ratio)
score        = clamp(round(raw), 0, 100)
```

Points clés	Détail
Normalisé par taille	Divisé par le nombre de fichiers → comparable entre projets
Critiques lourds	Pénalité proportionnelle en plus : <code>critical</code> pèse 5 et fait tomber la note (pénalité × nb critiques)
Lettres	A ≥ 90 , B ≥ 80 , C ≥ 70 , D ≥ 40 , F < 40 (grille 5 crans, reproduit « 49/100 D » du rapport de référence)
Domaine	<code>security</code> (injection/xss/secrets/security_misc/dependencies), <code>quality</code> (code_quality), <code>performance</code>

Le score est calculé sur **l'ensemble des findings dédupliqués, avant** filtrage `--severity-min` : la note reflète l'état réel du projet, pas ce qui est affiché dans le listing.

8. Codes de sortie

Code	Signification
0	Analyse terminée, aucune faille critique ou haute détectée.
1	Analyse terminée avec au moins une faille critique/haute , OU erreur d'exécution (dossier/repo introuvable, clonage échoué, règles illisibles, écriture impossible).
2	Erreur d' utilisation de la CLI (cible manquante, format <code>--output</code> invalide).

Le code 1 en présence de failles permet d'**interrompre un pipeline CI** (voir §13). Pour distinguer une vraie erreur d'une simple présence de failles, vérifiez la présence du message `[ERREUR]` dans la sortie stderr.

9. Référence des règles

9.1 Analyseur Python (AST) — fichier `.py`

ID	Sévérité	Détection
py-dangerous-eval	critical / high	eval() / exec() (critical si l'argument vient de input())
py-pickle-load	high	pickle.loads/load , marshal.loads/load , yaml.load
py-subprocess-shell	high	subprocess.*(..., shell=True)
py-sql-injection	high	requête SQL concaténée, f-string ou %
py-hardcoded-secret	high	variable nommée password / api_key / secret / token = littéral
py-os-system	high	os.system() / os.popen()
py-bare-except	medium	except: sans type
py-assert-security	medium	assert sur un contrôle de privilège (désactivé avec - 0)
py-dangerous-import	medium	import de pickle , marshal , shelve , telnetlib
py-long-function	low	fonction > 50 lignes
py-high-complexity	medium	complexité cyclomatique > 10
py-cognitive-complexity	medium	complexité cognitive > 15
py-deep-nesting	medium	imbrication > 4 niveaux
py-too-many-params	medium	plus de 4 paramètres
py-else-after-return	medium	bloc else après un return dans le if
py-redundant-boolean	low	x == True / x is False (préférer x)
py-empty-except	medium	bloc except sans aucune instruction
py-request-without-timeout	medium	requests.*() sans timeout (blocage possible, DoS)
py-verify-false	high	requests.*(..., verify=False) : vérification TLS désactivée
py-weak-hash	medium	hashlib.md5/sha1 (ou hashlib.new("md5"))
py-insecure-random	medium	random.* pour un jeton/clé/mot de passe (contexte sécurité)
py-xml-unsafe	high	parsing XML sans defusedxml importé (risque XXE/bombe XML)
py-tempfile-mktemp	medium	tempfile.mktemp() (nom prévisible, course)

9.2 Analyseur générique (regex) — multi-langages

ID	Langages	Sévérité	Détection
generic-sql-injection	php, js, ts, java, c#, ruby, go...	high	requête SQL construite dynamiquement
generic-xss-innerhtml	js, ts, html	high	<code>.innerHTML =</code>
generic-xss-dangerouslyset	js, ts	high	<code>dangerouslySetInnerHTML</code> (React)
generic-xss-document-write	js, ts	high	<code>document.write()</code>
generic-xss-vhtml	js, ts	medium	directive Vue <code>v-html</code>
generic-eval	js, ts, php	high	<code>eval()</code>
generic-php-unserialize	php	critical	<code>unserialize()</code>
generic-command-exec	php, ruby, bash	high	<code>shell_exec</code> , <code>system</code> , <code>passthru</code> , <code>popen</code> ...
generic-node-child-process	js, ts	high	<code>child_process.exec/spawn</code>
generic-node-child-process-import	js, ts	medium	import de <code>child_process</code>
generic-js-exec-call	js, ts	medium	appel <code>exec()</code> (à vérifier)
generic-java-runtime-exec	java, kotlin	high	<code>Runtime.exec</code> , <code>ProcessBuilder</code>
generic-secret-assignment	non-Python	high	secret affecté en dur
generic-todo-fixme	tous	low	<code>TODO</code> , <code>FIXME</code> , <code>HACK</code> , <code>XXX</code>
generic-cors-permissive	php, js, ts, python	high	<code>Access-Control-Allow-Origin: *</code>
generic-regex-dos	js, ts	high	<code>new RegExp(...)</code> sur valeur non-littérale (risque ReDoS)
generic-php-superglobal-input	php	medium	lecture brute de <code>\$_GET</code> / <code>\$_POST</code> / <code>\$_REQUEST</code>

generic-sql-select-star	sql, php, js	medium	requête <code>SELECT *</code> non ciblée
generic-weak-crypto	php, js, ts, java, c#, go, ruby...	medium	md5 / sha1 / DES / RC4 ... y compris <code>createHash("sha1")</code> et <code>MessageDigest.getInstance("MD5")</code>
generic-php-xxe	php	high	<code>simplexml_load_*</code> , <code>->loadXML</code> (risque XXE)
generic-path-traversal	php, js, ts, python, java...	medium	chemin de fichier construit par concaténation (<code>fopen("up/" . \$n)</code> , <code>readFileSync("d/" + f)</code>)
generic-php-insecure-rand	php	low	<code>rand()</code> / <code>mt_rand()</code> (préférer <code>random_int()</code>)

9.2b Analyseur de qualité (à accolades + Python)

Les IDs ci-dessous portent le préfixe du langage : `js-*` pour le

JavaScript, `ts-*` pour TypeScript, `php-*` , `java-*` , `csharp-*` , `go-*` ,

`kotlin-*` , `scala-*` , `rust-*` , `swift-*` , `c-*` , `cpp-*` ... (`py-*` sur

Python, via AST — voir §9.1). Seuils centralisés dans

`scanner/thresholds.py` .

Règle (préfixé)	Sév	Seuil	Détection
*-function-too-long	low	50 lignes	fonction trop longue
*-cognitive-complexity	medium	15	complexité cognitive élevée
*-high-complexity	medium	10	complexité cyclomatique élevée
*-deep-nesting	medium	4	imbrication excessive
*-too-many-params	medium	4	trop de paramètres
*-long-line	low	120	ligne trop longue
*-file-too-long	low	500	fichier trop long
*-redundant-boolean	low	—	<code>x == true</code> / <code>x === false</code>
*-loose-equality	low	—	<code>==</code> (préférer <code>===</code>)
*-else-after-return	medium	—	<code>else</code> après un <code>return</code>
*-empty-catch	medium	—	bloc <code>catch</code> vide (erreurs avalées)
*-commented-out-code	low	≥ 3 lignes	code commenté (probablement mort)
*-magic-number	low	—	nombre littéral non nommé (conservateur)
*-no-debug-console	low	—	<code>console.log/warn/error</code> (débogage)
*-no-alert	low	—	<code>alert</code> / <code>confirm</code> / <code>prompt</code>
*-deep-clone-json	medium	—	copie profonde via <code>JSON.parse(JSON.stringify(x))</code>
perf-blocking-sync-io	high	—	<code>readFileSync</code> / <code>execSync</code> ... bloquants (hors init/connexion)
perf-io-in-loop	medium	—	<code>await fetch/query</code> dans une boucle (N+1) hors <code>Promise.all</code>
perf-quadratic-loop	medium	—	<code>.includes()</code> / <code>indexOf</code> dans une boucle sur tableau ($O(n^2)$)

Les métriques de fonction JS/PHP/etc. sont calculées par équilibrage

d'accolades sur un code masqué (chaînes/commentaires/templates) : les

valeurs sont **indicatives**. Les anti-faux-positifs ignorent les appels

dans les fonctions `config / init / load`, `Promise.all`, `for await`, les

caches `Map / Set` et les littéraux de constantes.

9.3 Détecteur de secrets

ID	Sévérité	Détection
secret-aws-access-key	critical	AKIA... / ASIA... (20 car.)
secret-aws-secret-key	critical	<code>aws_secret_access_key = "...40 car..."</code>
secret-github-token	critical	<code>ghp_...</code> , <code>github_pat_...</code>
secret-stripe	critical	<code>sk_live_...</code> , <code>rk_live_...</code>
secret-private-key	critical	bloc <code>-----BEGIN ... PRIVATE KEY-----</code>
secret-google-api	high	AIza... (39 car.)
secret-slack-token	high	<code>xox[baprs]-...</code>
secret-npm-token	high	<code>npm_...</code>
secret-bearer	high	Bearer <jeton>
secret-slack-webhook	high	URL <code>hooks.slack.com/services/...</code>
secret-jwt	medium	JWT <code>eyJ...</code>
secret-high-entropy	low	chaîne ≥ 16 car. à forte entropie (Shannon $\geq 4,5$)
config-plaintext-secret	high	secret en clair dans <code>.env / .json / .yaml / .ini</code>

9.4 Dépendances

ID	Sévérité	Détection
dep-cve	de critical à low	CVE d'un paquet via OSV.dev (sévérité déduite du score CVSS ou du niveau GHSA)

Manifestes supportés : `requirements*.txt`, `package.json`, `composer.json`,

`Gemfile`, `go.mod`, `Pipfile`, `Cargo.toml`, `pom.xml`.

9.5 Catégories

`injection` · `secrets` · `xss` · `code_quality` · `security_misc` ·

performance · dependencies

Ces catégories sont regroupées en **domaines** de score : security

(injection, secrets, xss, security_misc, dependencies), quality

(code_quality), performance .

10. Personnalisation des règles

Toutes les règles **regex** vivent dans `rules/patterns.json` : vous pouvez en ajouter, modifier ou retirer **sans toucher au code**.

Schéma d'une entrée generic (source)

```
{
  "id": "generic-hash-faible",
  "category": "security_misc",
  "severity": "medium",
  "languages": ["php", "javascript"],
  "pattern": "(?i)\\b(md5|sha1)\\s*\\(",
  "title": "Fonction de hachage faible",
  "description": "md5/sha1 ne sont pas sûrs pour les mots de passe.",
  "recommendation": "Utiliser bcrypt, argon2 ou au minimum sha256 avec sel."
}
```

Schéma d'une entrée secrets

```
{
  "id": "secret-nouvelle-cle",
  "name": "Clé privée NouveauService",
  "category": "secrets",
  "severity": "critical",
  "pattern": "\\bNS_[A-Za-z0-9]{24}\\b",
  "title": "Clé NouveauService exposée",
  "description": "Une clé d'API NouveauService est visible dans le code.",
  "recommendation": "Révoquer la clé et utiliser l'environnement."
}
```

Champs disponibles

Champ	Règles <code>generic</code>	Règles <code>secrets</code>	Obligatoire
id	✓	✓	oui
category	✓	✓	oui
severity	✓	✓	oui
pattern (regex)	✓	✓	oui, sauf règles <code>builtin</code>
languages (liste)	✓	X (scan global)	non
title	✓	✓	oui
description	✓	✓	recommandé
recommendation	✓	✓	recommandé
builtin	✓	✓	non (entropy pour les secrets)

Règles `builtin` (calculées par le code)

Une seule règle reste « builtin » dans le stock : `entropy` (détection des chaînes à forte entropie dans la section `secrets`). Elle n'a pas de `pattern` mais un champ `builtin: "entropy"` .

Les métriques de fonction (longueur, complexité, imbrication...) ne sont **plus** des patterns : elles sont calculées directement par `scanner/python_analyzer.py` (AST, exact) et `scanner/quality_analyzer.py` (accolades, heuristique). Réglez leurs seuils dans `scanner/thresholds.py` (le JSON de règles ne les contient pas).

Validation : CodeScan ignore une règle dont la regex est invalide et

l'indique dans la console (`pattern invalide`), sans interrompre l'analyse.

11. Exécution des tests

```
cd codescan
python -m unittest discover tests -v
```

La suite (~**240 tests**) couvre :

- chaque règle de l'analyseur Python sur du code vulnérable **et** du code sain (anti-faux-positifs), y compris les nouvelles règles de qualité (complexité cognitive, imbrication, paramètres, `else` après `return` ...) ;
 - les règles regex génériques (SQL, XSS, eval, TODO, CORS, ReDoS...) ;
 - l'analyseur de qualité (métriques de fonction JS/PHP, lignes longues, async/perf, anti-faux-positifs `Promise.all` /config) ;
 - le détecteur de secrets (AWS, GitHub, Stripe, entropie, placeholders) ;
 - le crawler (dossiers ignorés, binaires, détection de langage) ;
 - le parsing des manifestes et le calcul des scores CVSS ;
 - le calcul du score /100 (calibration sur la référence Herald → 49/100 D, bandes A-F, domaines) et du rapport HTML (hero, cartes, niveaux, échappement, CSS print) ;
 - le filtrage par sévérité.
-

12. Dépannage et FAQ

« `python` n'est pas reconnu » (Windows)

Le lanceur `py` est installé avec Python : utilisez `py main.py ...`.

Sinon, désactivez l'alias « App execution alias » dans les paramètres Windows.

« **git clone a échoué : Repository not found** »

L'URL est invalide ou le dépôt est privé. CodeScan ne gère **pas**

l'authentification : utilisez un dépôt public ou clonez-le d'abord

manuellement puis analysez-le avec `--path`.

« **OSV.dev indisponible** »

Le réseau est coupé ou le service est en maintenance. CodeScan l'indique dans

la console et **continue** l'analyse sans la partie dépendances. Utilisez

`--no-deps` pour un fonctionnement hors ligne sans message d'erreur.

« **Le rapport affiche beaucoup de faux positifs** »

- Les heuristiques (`generic-long-function` , `generic-high-complexity` , `secret-high-entropy`) sont des **indicateurs**, pas des certitudes : vérifiez chaque résultat.
- Réduisez le bruit avec `--severity-min=medium` OU `--severity-min=high` .

« Un fichier n'est pas analysé »

Vérifiez qu'il n'est pas dans un dossier ignoré (`node_modules/` , `.git/` , `venv/` , `__pycache__/` ...), qu'il ne fait pas plus de 2 Mo et que son extension est reconnue (voir `LANG_BY_EXT` dans `scanner/crawler.py`).

« Les caractères accentués sont mal affichés dans la console »

L'affichage est forcé en UTF-8 par CodeScan. Si le terminal ne le supporte pas, définissez `PYTHONIOENCODING=utf-8` ou activez le terminal UTF-8.

« Comment supprimer les secrets de l'historique git ? »

CodeScan les détecte, mais il ne les efface pas. Après correction du code, régénérez l'historique (`git filter-repo`), **révoquez** les secrets exposés sur le service concerné, puis ajoutez les fichiers à `.gitignore` .

13. Intégration CI/CD

Exemple d'étape GitHub Actions :

```
- name: Analyser le code avec CodeScan
  run: |
    pip install -r requirements.txt
    python main.py --path . --output codescan.json --severity-min=high
    # Le workflow échoue si des failles haute/critique sont détectées
    # (code de sortie 1).
- name: Archiver le rapport
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: codescan-report
    path: codescan.json
```

Le rapport JSON est alors exploitable par d'autres étapes (diff de résultats, notification, tableau de bord).

14. Limites connues

1. **Heuristiques approximatives** — la longueur et la complexité des fonctions (JS/PHP/Java/C#/Go...) reposent sur un équilibrage d'accolades : à confirmer par un humain.
1. **Contraintes de version « plage »** — une dépendance déclarée avec `>=` ou `^` est vérifiée par sa **version minimale** : une CVE introduite plus tard dans la plage peut être manquée.
1. **Secrets par entropie** — `secret-high-entropy` produit des résultats potentiellement faux : toujours vérifier avant suppression.
1. **Pas d'analyse sémantique multi-fichiers** — CodeScan analyse fichier par fichier : un flux de données traversant plusieurs fichiers (une entrée utilisateur « sale » utilisée loin de sa source) n'est pas suivi.
1. **Encodage** — les fichiers non-UTF-8 sont relus en latin-1 : l'analyse reste possible mais les extraits (`snippet`) peuvent être dégradés.

15. Mode IRON MAN AI — audit Kali (sites web)

L'outil de web audit s'appelle **IRON MAN AI** et s'utilise depuis une machine **Kali Linux**. Il pilote **tous les outils de sécurité disponibles** contre un site web et **réunit les failles** dans le même style de rapport (HTML /100, JSON ou PDF). C'est un **module séparé** de `main.py` : il ne modifie pas le comportement de l'analyse statique.

15.1 LA commande unique — audit complet en PDF

Une **seule commande** lance **tout l'audit ensemble** : tous les outils (web **et** invasifs), l'un après l'autre, **au maximum, sans limite de temps d'analyse**, avec toutes les autorisations requises, et produit

l'audit complet en PDF (plus JSON et HTML) :

```
uv run python ironman.py --url https://exemple.com --authorized
```

C'est l'équivalent de

```
kali_scan.py --url ... --authorized --full --attack --pdf : mode maximal
```

(wordlist complète, `nmap -p- -sC`, threads élevés, **aucun timeout**, `sqlmap --level 3 --risk 3`) et rapport PDF 100 % stdlib (aucun navigateur requis). Utilisez `--tools` / `--exclude` / `--dry-run` pour restreindre, et `--check` d'abord pour vérifier que tout est installé.

15.2 Préflight (`--check`)

Vérifie la présence de chaque binaire (`nmap`, `nikto`, `whatweb`, `gobuster`, `dirsearch`, `ssllscan`, `nuclei`, `wafw00f`, `dnsrecon` ...).

Pour tout outil manquant, il affiche une commande

```
sudo apt-get install -y <paquet> exacte à copier.
```

```
uv run python kali_scan.py --check          # outils web présents ?
uv run python kali_scan.py --check --attack  # + sqlmap, xssstrike, commix, hydra
```

15.3 Scan normal (`kali_scan.py`)

Le flag `--authorized` est **obligatoire** (confirme que la cible est autorisée). Les outils **invasifs** sont exclus par défaut et ne tournent qu'avec `--attack` (hydra exige en plus les wordlists `--hydra-users` / `--hydra-passwords`).

```
uv run python kali_scan.py --url https://exemple.com --authorized \
  --output rapport_web.html
```

Options utiles : `--dry-run` (affiche les commandes sans exécuter),

```
--tools nmap,nuclei / --exclude nmap (filtrer), --allow-missing
```

(continuer malgré un outil manquant), `--keep-tmp` (conserver les logs

bruts par outil), `--pdf` (produire aussi le rapport PDF). Le **code de sortie** est identique au mode statique : 0 aucun relevé critique/haute,

1 sinon, 2 erreur d'usage.

Un **serveur local volontairement vulnérable** est fourni pour s'initier en toute sécurité :

```
uv run python examples/vuln_server.py --port 8123
uv run python kali_scan.py --url http://127.0.0.1:8123 --authorized \
  --output rapport_web.html --allow-missing
```

 **Manuel complet (Kali)** : [docs/MANUEL_KALI.md](#) · **Manuel Windows** :

[docs/MANUEL_WINDOWS.md](#) . Le manuel Kali se convertit en PDF via

[docs/make_pdf.py](#) (Chrome/Edge headless, sinon « Imprimer en PDF »).

Documentation générée pour IRON MAN AI (CodeScan) 1.4.0. Le README contient une version condensée ; ce manuel fait référence pour l'usage détaillé.

Généré par CodeScan — manuel imprimable.

IRON MAN AI — Audit web (Kali)

Manuel d'utilisation — IRON MAN AI sur Kali Linux

L'outil d'audit web s'appelle **IRON MAN AI** et s'utilise sur une machine

Kali Linux : vérifier que tous les outils de sécurité sont présents,

les installer si nécessaire, puis lancer un **audit de sécurité d'un site web**

avec l'ensemble des outils disponibles (nmap, nikto, whatweb, gobuster,

dirsearch, sslscan, nuclei, wafw00f, dnsrecon) et, si vous y êtes

explicitement autorisé, les outils d'attaque sqlmap, xssstrike, commix et

hydra.

Une seule commande (`ironman.py`) lance **tout l'audit ensemble** :

tous les outils, l'un après l'autre, **au maximum**, sans limite de temps

d'analyse, et produit **l'audit complet en PDF** (ainsi qu'en JSON et en HTML).

⚠ Sécurité et éthique. N'analysez que des sites que **vous possédez**

ou pour lesquels vous disposez d'une **autorisation écrite** du

propriétaire. Le flag `--authorized` est obligatoire : il confirme que

vous comprenez cette règle. Les agissements malveillants sont interdits.

1. Prérequis

- **Python 3.10+** (le cœur est en stdlib, aucune dépendance).
- **Kali Linux** (les outils ci-dessous y sont généralement présents).
- Les outils suivants, installés comme **paquets Kali** :

Outil	Paquet apt	But
nmap	nmap	Découverte des ports/services
nikto	nikto	Scanner de vulnérabilités web
whatweb	whatweb	Détection des technologies
gobuster	gobuster	Énumération de répertoires
dirsearch	dirsearch	Énumération de chemins web
ssllscan	ssllscan	Contrôle TLS/SSL (https)
nuclei	nuclei	Template de vulnérabilités
wafw00f	wafw00f	Détection de WAF
dnsrecon	dnsrecon	Reconnaissance DNS

Et pour le mode **attack** : `sqlmap`, `xsstrike`, `commix`, `hydra`.

2. Vérifier les outils présents avant de lancer (PRÉFLIGHT)

La commande dédiée est `--check`. Elle examine chaque binaire et,

s'il en manque un, **affiche la commande exacte à exécuter** pour l'installer.

```
cd /chemin/codescan
uv run python kali_scan.py --check
```

Sortie exemple :

```
=== Préflight IRON MAN AI (Kali) ===
MANQUANT nmap      nmap -> sudo apt-get install -y nmap
MANQUANT nikto     nikto -> sudo apt-get install -y nikto
... (tous les outils manquants listés) ...
5/9 outils présents, 4 manquant(s).
```

On peut vérifier aussi les outils "attack" avec `--check --attack`.

Pour **installer tous les outils manquants en une seule fois**, copiez la commande affichée (elle est construite automatiquement) :

```
sudo apt-get update && sudo apt-get install -y nmap nikto whatweb gobuster dirsearch ss
# Si vous voulez aussi le mode attack :
sudo apt-get install -y sqlmap xssstrike commix hydra
```

Le scanner **ne lance plus le scan** s'il manque des outils « web »

essentiels, afin de ne jamais donner un faux sentiment de couverture.

Ajoutez `--allow-missing` si vous voulez tout de même lancer avec les

outils présents (le rapport signale alors les outils manquants).

3. LA commande unique : audit complet en PDF (IRON MAN AI)

Sur Kali, une **seule commande** fait **tout l'audit ensemble** : elle

lance **tous les outils les uns après les autres** (web **et** invasifs),

au maximum de leurs capacités, sans limite de temps d'analyse, avec

toutes les autorisations requises, et produit **l'audit complet en PDF**

(plus le JSON et le HTML) :

```
uv run python ironman.py --url https://example.com --authorized
```

C'est l'équivalent exact de

```
kali_scan.py --url ... --authorized --full --attack --pdf
```

 . Concrètement,

le drapeau `--full` (alias `--maximal`) :

- active `--attack` (sqlmap, xssstrike, commix, hydra) ;
- **supprime les timeouts** : chaque outil tourne jusqu'à son terme ;
- utilise la **wordlist complète** (plus de plafond de 200 mots) ;
- pousse les outils au maximum : `nmap -sV -Pn -p- -sC` (tous les ports, scripts), `sqlmap --level 3 --risk 3`, threads élevés ;
- **produit automatiquement le PDF** de l'audit complet, à côté des rapports JSON et HTML — écrits par défaut dans le **dossier central** `rapports/` (`rapports/audit_web_AAAAMMJJ_HHMMSS.json` + `.html` + `.pdf`, horodatés). `-o /chemin/rapport` pour écrire ailleurs.

```
uv run python ironman.py --url https://example.com --authorized
```

Le PDF est généré **100 % en bibliothèque standard** (aucun navigateur requis) : en-tête, score global, synthèse par sévérité, résultat par outil, préflight et détail complet des relevés groupés par outil.

Version **raisonnable** (outils web uniquement, timeouts courts, wordlist bornée, **sans** PDF) : utilisez `kali_scan.py --url ... --authorized`

(voir section 4). Vous pouvez aussi restreindre `ironman.py` avec

`--tools`, `--exclude` ou `--dry-run`, et vérifier d'abord avec `--check`.

4. Lancer un scan web (toutes les failles)

Le scan lance **tous les outils disponibles** (palier « web », non invasif) contre la cible, avec **timeouts limites et wordlists réduites** pour

rester raisonnable, puis **réunit les failles trouvées**.

```
# Cible http
uv run python kali_scan.py --url http://example.com --authorized

# Cible https (sslscan est alors actif)
uv run python kali_scan.py --url https://shop.example.com --authorized

# Avec un rapport (+ PDF de l'audit complet si --pdf)
uv run python kali_scan.py --url https://example.com --authorized \
  --output rapport_web.html
uv run python kali_scan.py --url https://example.com --authorized \
  --output rapport_web.json
uv run python kali_scan.py --url https://example.com --authorized \
  --output rapport_web.html --pdf
```

Un **serveur de test local vulnérable** est fourni (cible sûre pour s'initier) :

```
uv run python examples/vuln_server.py --port 8123
uv run python kali_scan.py --url http://127.0.0.1:8123 --authorized \
  --output rapport_web.html --allow-missing
```

Options principales

Option	Effet
<code>--url URL</code>	Cible à scanner (http/https)
<code>--check</code>	Préflight (présence des outils) sans scanner
<code>--authorized</code>	Confirme l'autorisation (obligatoire pour scanner)
<code>--attack</code>	Ajoute les outils invasifs (sqlmap, xssstrike, commix, hydra)
<code>--full / --maximal</code>	Mode maximal : tout (web et attack), aucun timeout, wordlist complète, nmap -p- -sC , audit complet (JSON + HTML + PDF)
<code>--pdf</code>	Produit aussi le rapport PDF de l'audit complet
<code>--dry-run</code>	Affiche les commandes qui seraient lancées, rien n'est exécuté
<code>--tools nmap,nuclei</code>	Limite aux outils listés
<code>--exclude nmap</code>	Exclut des outils
<code>--output fichier.html</code>	Rapport écrit (sinon résumé console)
<code>--allow-missing</code>	Continuer si des outils web manquent
<code>--tool-timeout SECONDES</code>	Borne chaque outil (même en mode maximal), ex. <code>--tool-timeout 600</code> = 10 min/outil
<code>--verbose</code>	Détail du préflight

5. Le mode attack (que pour les cibles autorisées)

Les outils invasifs sont exclus par défaut. Ajoutez `--attack` quand vous avez l'autorisation explicite :

```
uv run python kali_scan.py --url https://example.com --authorized --attack
```

- sqlmap, xssstrike, commix tournent en mode **détection** (pas de dump / pas de destruction de données).
- hydra est **désactivé par défaut** sauf si vous fournissez des wordlists (objectif : ne jamais brute-forcer sans information préalable).

6. Comprendre les sorties

Chaque outil produit des **relevés** (findings) associés à la cible. Le rapport HTML affiché une **note de dette web /100** (parité avec le rapport statique), des cartes (Relevés, Critiques, À revoir, MINEUR, Outils OK), la **table par outil** (statut, durée, nombre de relevés), le **préflight** et les relevés **groupés par outil**. Le JSON reproduit la même structure (méta, résumé, findings, score).

Le **code de sortie** de `kali_scan.py` :

- `0` : aucun relevé critique/haute ;
- `1` : au moins un relevé **critique ou haute** (utile en CI) ;
- `2` : erreur d'usage (pas d'URL, pas d' `--authorized` , outils manquants).

7. Aller plus loin

- **Mode normal (`kali_scan.py`)** : wordlists **limitées** (max 200 mots) et **timeouts** par outil (120 à 480 s) pour rester raisonnable ; un outil qui timeout n'arrête pas le scan. C'est un premier passage reproductible.
- **Mode maximal (`ironman.py` / `--full`)** : **aucune limite** — wordlist complète, aucun timeout, outils poussés au maximum et audit complet en PDF. C'est la commande unique « tout l'audit ensemble ».
- Les **logs bruts** de chaque outil sont écrits dans un répertoire temporaire (`codescan-kali/TIMESTAMP/`) pour analyse manuelle.

8. Dépannage

Problème	Cause probable	Solution
2 erreur d'usage	Pas de <code>--authorized</code>	Ajouter <code>--authorized</code>
Outil «manquant » à <code>--check</code>	Binaire absent	<code>sudo apt-get install -y <paquet></code>
<code>--attack</code> n'effectue rien	hydra a besoin de wordlists	Fournir <code>--hydra-users</code> / <code>--hydra-passwords</code>
sslsan non lancé	Cible en http	Utiliser une URL https

Rappel final : IRON MAN AI (CodeScan) est un outil de **défense**.

Ne l'utilisez que pour des sites que vous possédez ou que vous êtes autorisé à auditer. La sécurité est une histoire de confiance, pas de parcours d'outils.

Généré par CodeScan — manuel imprimable.

IRON MAN AI — Pentest WiFi

Manuel d'utilisation — IRON MAN AI : Audit WiFi

Le module **WiFi** de IRON MAN AI permet de tester la sécurité d'un réseau WiFi que vous possédez (ou pour lequel vous avez une autorisation écrite) : scan des réseaux à portée, capture et crack du handshake WPA2, audit WPS, recommandations de durcissement.

 **Sécurité et éthique.** Ce module ne s'utilise que sur votre propre

réseau ou sur un réseau que vous êtes **explicitement autorisé** à

tester (contrat de pentest, réseau d'entreprise mandaté...). Le flag

```
--authorized
```

 est obligatoire.

Cracker le WiFi d'un inconnu est illégal et n'est pas couvert par

cet outil. Le test de déauthentification perturbe le réseau : faites-le

uniquement sur votre matériel et prévenez les utilisateurs.

1. Prérequis

- **Python 3.10+** (le cœur est en stdlib, aucune dépendance).
- **Kali Linux** avec une **carte WiFi compatible mode monitor**.
- Les outils **aircrack-ng** (airmon-ng, airodump-ng, aireplay-ng, aircrack-ng) : `sudo apt-get install -y aircrack-ng`
- Optionnel : `reaver` / `bully` / `pixiewps` (audit WPS), `wash`, `hashcat` ou `john` (crack accéléré).

Vérifiez la présence des outils (aucune action sur le réseau) :

```
cd /chemin/codescan
python mobile_scan.py --wifi --check
```

2. Première utilisation : scanner les réseaux à portée

```
# Lister les interfaces WiFi disponibles
iw dev

# Lancer le scan (30 s par défaut)
sudo python mobile_scan.py --wifi --interface wlan0 --authorized
```

Le script active le **mode monitor** sur l'interface, scanne pendant la durée demandée puis affiche les réseaux détectés, triés par force de signal :

1. MonReseau	AA:BB:CC:DD:EE:FF	ch 6	-45 dBm	WPA2
2. Voisin_WPA	11:22:33:44:55:66	ch 11	-70 dBm	WPA

- `--scan-time N` : durée du scan en secondes (30 par défaut).
- L'interface est **automatiquement restaurée** à la fin (mode managed, NetworkManager relancé).



Conseil d'utilisation efficace : lancez le scan en premier, puis

relevez le **BSSID** et le **canal** du réseau que vous voulez tester

(le vôtre). Ces deux valeurs servent aux étapes suivantes.

3. Tester la robustesse du mot de passe : capture + crack WPA2

La méthode standard pour vérifier qu'un mot de passe WiFi n'est pas faible :

```
sudo python mobile_scan.py --wifi --interface wlan0 \  
--bssid AA:BB:CC:DD:EE:FF --ssid MonReseau --channel 6 \  
--crack --authorized
```

Ce que fait la commande :

1. **Capture ciblée** : airodump-ng écoute le canal du réseau cible.
1. **Déauthentification** : un client connecté est déconnecté (5 paquets deauth) pour forcer sa reconnexion — c'est pendant cette reconnexion que le handshake WPA2 est capturé.
1. **Crack par dictionnaire** : aircrack-ng essaie chaque mot de passe de la wordlist (rockyou par défaut si présente) contre le handshake.

Options utiles :

Option	Effet
--capture-time N	Durée de capture (120 s par défaut)
-w /chemin/wordlist.txt	Dictionnaire à utiliser
--method john / --method hashcat	Autre outil de crack
--wps	Ajoute un test de vulnérabilité WPS (reaver)

Si le mot de passe est trouvé, il s'affiche :

```
[wifi] 🎉 MOT DE PASSE TROUVÉ : motdepassefaible123
```

💡 **Conseil d'utilisation efficace** : si la capture échoue

(« Aucun handshake »), c'est qu'aucun client ne s'est reconnecté.

Relancez avec un client actif (téléphone en veille se reconnecte

souvent tout seul) ou augmentez `--capture-time` . Pour vos tests,

utilisez une wordlist adaptée (rockyou pour les mots de passe

courants, ou une wordlist maison avec vos anciens mots de passe).

4. Audit WPS

Le WPS (Wi-Fi Protected Setup) est une porte d'entrée connue : un PIN faible ou une implémentation buguée permet de retrouver la clé WPA2 sans dictionnaire.

```
sudo python mobile_scan.py --wifi --interface wlan0 \  
--bssid AA:BB:CC:DD:EE:FF --channel 6 --wps --authorized
```

- `reaver` tente les PIN possibles (peut prendre du temps — borné à 30 min par tentative).
- Si le WPS est vulnérable, le PIN (et parfois le mot de passe) s'affiche.
- Si le routeur **verrouille** le WPS après plusieurs essais, l'outil le signale.

 **Conseil** : l'audit WPS est bruyant (le routeur peut se verrouiller

ou se redémarrer). À n'utiliser que sur votre propre équipement, en

dehors des heures d'usage.

5. Simuler sans rien casser : le mode dry-run

Avant chaque vraie campagne, visualisez **exactement** les commandes qui seraient exécutées :

```
python mobile_scan.py --wifi --bssid AA:BB:CC:DD:EE:FF --crack --wps \  
--authorized --dry-run
```

Aucun paquet n'est émis : utile pour vérifier la ligne de commande et pour la documentation.

6. Rapport de l'audit

Ajoutez `-o rapport` pour produire un **rapport JSON + HTML** consolidé

(réseaux, handshake, crack, WPS, recommandations) :

```
sudo python mobile_scan.py --wifi --bssid AA:BB:CC:DD:EE:FF --crack \  
  --authorized -o /tmp/rapport_wifi \  
# → /tmp/rapport_wifi.json + /tmp/rapport_wifi.html
```

Le rapport se termine par des **recommandations de sécurité** : mot de passe plus long, WPS désactivé, WPA3 si disponible, désactivation du SSID broadcast, etc.

7. Bonnes pratiques pour une utilisation efficace

1. **Cadre légal d'abord** : réseau possédé ou autorisation écrite. Documentez le périmètre avant de commencer.
 1. **Interface propre** : utilisez une carte USB dédiée (Alfa AWUS036ACH ou similaire) — les cartes internes supportent mal le mode monitor.
 1. **Antenne et position** : rapprochez-vous du routeur testé pour un signal stable (> -70 dBm) : le handshake se capture beaucoup plus vite.
 1. **Wordlists adaptées** : rockyou pour les mots de passe courants ; pour tester la politique d'un réseau, créez une wordlist dédiée (années, prénoms, marques...).
 1. **Minimisez l'impact** : les deauth et reaver perturbent le réseau — testez en dehors des heures d'utilisation.
 1. **Vérifiez les recommandations** : changez le mot de passe, désactivez WPS, passez en WPA3/WPA2-AES (jamais TKIP), activez le filtrage des invités.
-

8. Dépannage

Problème	Cause probable	Solution
Interface monitor non active	mode monitor non démarré	Vérifiez que vous êtes root (<code>sudo</code>) et que la carte supporte le monitor (<code>iw list</code>)
Aucun réseau détecté	mauvaise interface ou canal	Relancez avec <code>--interface</code> correct ; essayez <code>airodump-ng <iface></code> à la main
Aucun handshake	aucun client actif	Attendez une reconnexion, augmentez <code>--capture-time</code>
Aucune wordlist trouvée	pas de wordlist Kali	Installez rockyou : <code>sudo apt-get install -y wordlists</code> puis décompressez <code>/usr/share/wordlists/rockyou.txt.gz</code>
WPS verrouillé	trop de tentatives	Patiencez (verrou temporaire) ; le test WPS est à faire une seule fois

Généré par CodeScan — manuel imprimable.

IRON MAN AI — Analyse Android (APK)

Manuel d'utilisation — IRON MAN AI : Analyse Android

Le module **Android** de IRON MAN AI réalise une analyse statique de sécurité d'un fichier APK/AAB (application Android) : permissions, composants exportés, secrets codés en dur, code à risque (WebView, crypto faible, SQL, TLS...), URLs, le tout sans décompiler entièrement l'application.

 **Sécurité et éthique.** N'analysez que des applications que vous

avez développées, que vous possédez, ou pour lesquelles vous avez

une **autorisation écrite** (audit interne, test d'intrusion mandaté).

Le flag `--authorized` est obligatoire. Analyser une application que vous ne possédez pas (pour voler des secrets ou contourner des protections) est illégal et n'est pas couvert par cet outil.

1. Prérequis

- **Python 3.10+** (le cœur est en stdlib, aucune dépendance).
- L'analyse de base (manifeste binaire, chaînes dex, secrets) fonctionne **sans aucun outil externe** : le manifeste Android est un format binaire (AXML) décodé nativement par le module.
- **Optionnel** (analyse enrichie) : - `apktool` : décode le manifeste et les ressources (`sudo apt-get install -y apktool`) - `jadx` : décompilation complète du code Java (`sudo apt-get install -y jadx`)

Vérifiez les outils disponibles :

```
cd /chemin/codescan
python mobile_scan.py --android --check
```

2. Première analyse : un APK en une commande

```
python mobile_scan.py --android --apk /chemin/app.apk --authorized
```

Le module affiche d'abord les outils disponibles, puis le **résumé** de l'analyse :

```
Package      : com.example.app
Version      : 2.4.1 (code 241)
minSdk/targetSdk : 23 / 34
SHA-256      : 3f2a9c1d...
```

```
Total findings : 12
```

```
high    4  █████
medium  6  ███████
low     2  █
```

```
Permissions dangereuses : 3
```

```
Composants exportés : 2
```

```
Secrets détectés : 1
```

Puis le détail des findings, classés par sévérité.

3. Ce que l'outil détecte

Manifeste (AndroidManifest.xml)

- **Permissions dangereuses** : CAMERA, RECORD_AUDIO, LOCALISATION, SMS, contacts... (protection level *dangerous*).
- **Permissions à haut risque** : QUERY_ALL_PACKAGES, MANAGE_EXTERNAL_STORAGE, SYSTEM_ALERT_WINDOW, REQUEST_INSTALL_PACKAGES...
- **Composants exportés** : activity/service/receiver/provider invocables par d'autres applications.
- **Debuggable** (`android:debuggable="true"`), **backup autorisé** (`android:allowBackup="true"`), **trafic en clair** (`android:usesCleartextTraffic="true"`).
- **minSdkVersion obsolète** (< 21, Android 5.0).

Bytecode (classes*.dex)

- **Secrets codés en dur** : clés AWS (AKIA...), clés API Google (Alza...), Stripe, tokens GitHub/Slack/Firebase, clés privées PEM, `password=...` / `api_key=...` dans le code.
- **Code à risque** : WebView avec JavaScript, `addJavascriptInterface`, hachage MD5/SHA-1, chiffrement faible (DES/RC4/AES-ECB), requêtes SQL concaténées, validation TLS désactivée, accès aux identifiants de l'appareil (IMEI...), chargement dynamique de code.
- **URLs** : endpoints HTTP(S) embarqués, signalés s'ils utilisent HTTP en clair.

4. Produire un rapport HTML/JSON

```
python mobile_scan.py --android --apk app.apk --authorized -o /tmp/rapport_app  
# → /tmp/rapport_app.json + /tmp/rapport_app.html
```

- `-o rapport.html` : HTML seul ; `-o rapport.json` : JSON seul.
- Le rapport HTML est un document autonome et lisible dans un navigateur (findings triés par sévérité, permissions, composants, secrets, URLs).

5. Options utiles

Option	Effet
<code>--no-secrets</code>	Désactive la détection de secrets
<code>--no-code</code>	Désactive l'analyse du bytecode (dex)
<code>--jadx</code>	Enrichit l'analyse par décompilation jadx (lent : ~10 min sur une grosse app)
<code>-v</code> / <code>--verbose</code>	Affiche le détail complet des findings
<code>--dry-run</code>	Affiche les étapes sans rien analyser

6. Aller plus loin avec apktool / jadx

Si `apktool` est installé, le manifeste binaire est décodé par apktool

(analyse plus fidèle des ressources). Si `jadx` est installé, ajoutez

`--jadx` pour lancer une **décompilation complète** en complément :

les mêmes patterns de code à risque sont recherchés dans le code Java

décompilé, avec le fichier et la ligne exacts.

⚠ La décompilation jadx d'une application réelle (milliers de classes)

prend ~**10 minutes** et consomme beaucoup de RAM. Elle est **opt-in** :

sans `--jadx`, l'analyse (manifeste + dex + secrets) prend quelques

secondes.

Installation :

```
sudo apt-get update  
sudo apt-get install -y apktool jadx
```

7. Interpréter les résultats

- **high / critical** : à corriger en priorité — secret exposé, trafic en clair, composant exporté sensible, TLS désactivé.
- **medium** : à corriger selon le contexte — hachage faible, backup autorisé, minSdk obsolète.
- **low** : bonnes pratiques — permissions dangereuses justifiées, composants exportés intentionnels.

Chaque finding contient une **recommandation** précise (utiliser le Android Keystore, passer en AES-GCM, épingler les certificats, paramétrer les requêtes SQL...).

8. Bonnes pratiques pour une utilisation efficace

1. **Cadre légal d'abord** : app possédée ou autorisation écrite.
1. **Comparez les versions** : analysez l'APK de production ET un build de dev — les secrets et le flag debuggable s'y glissent souvent.
1. **Vérifiez les faux positifs** : un mot de passe dans un exemple de documentation embarqué n'est pas forcément un secret réel — confirmez dans le code décompilé.
1. **Intégrez à la CI** : lancez l'analyse à chaque build pour détecter les secrets avant la mise en production.
1. **Ne stockez jamais de secrets** : API keys dans le code = exposées (l'APK est téléchargeable par n'importe qui). Utilisez le backend et le Android Keystore.

9. Dépannage

Problème	Cause probable	Solution
Fichier introuvable	mauvais chemin	Vérifiez le chemin de l'APK (ls -la)
AndroidManifest.xml non lisible	APK corrompu ou non-Android	Vérifiez avec file app.apk (doit être un zip)
Aucun finding	app bien sécurisée... ou analyse incomplète	Installez apktool/jadx pour l'analyse approfondie
--apk requis	mode Android sans fichier	Ajoutez --apk <fichier.apk>

Généré par CodeScan — manuel imprimable.

IRON MAN AI — Audit de périphérique

Manuel d'utilisation — IRON MAN AI : Audit de périphérique Android

Le module **Périphérique** de IRON MAN AI audite la **posture de sécurité** d'un appareil Android (téléphone, tablette) que vous possédez : type de verrou d'écran, chiffrement du stockage, débogage USB, adb réseau, bootloader/OEM unlock, SELinux, fausses positions, services d'accessibilité, sources inconnues. Il produit une **note de posture /100** et des recommandations de durcissement.

Toutes les commandes sont **en lecture seule** (getprop, settings get, dumpsys, getenforce) : **aucune modification de l'appareil**.

 **Sécurité et éthique.** Cet audit teste la *résistance* de votre

appareil aux techniques de déverrouillage et de compromission. Il ne

permet **pas** de déverrouiller un appareil que vous ne possédez pas.

Le flag --authorized est obligatoire : votre matériel ou un appareil

pour lequel vous avez une autorisation écrite (audit interne, test mandaté). Tenter de contourner le verrou d'un appareil sans autorisation est illégal.

1. Prérequis

- **adb** : `sudo apt-get install -y adb` (OU `android-tools-adb`).
- Un appareil Android **branché en USB** avec **débogage USB activé** : 1. Paramètres → À propos → tapez 7 fois sur « Numéro de build » (active les options développeur) ; 2. Paramètres → Options développeur → activez **Débogage USB** ; 3. À la première connexion, acceptez l'invite « Autoriser le débogage USB ? » sur l'écran de l'appareil (cochez « Toujours autoriser »).
- **Important** : activer le débogage USB nécessite l'accès à l'écran de l'appareil — c'est la garantie que c'est le vôtre.

Vérifiez les outils :

```
cd /chemin/codescan
python mobile_scan.py --check          # vérifie les 3 modules (WiFi, Android, adb)
python mobile_scan.py --device --check # vérifie uniquement adb
```

2. Première analyse : un appareil en une commande

```
python mobile_scan.py --device --authorized
```

- Si un **seul** appareil est connecté, il est détecté automatiquement.
- Si **plusieurs** sont branchés, précisez le serial (`adb devices` pour le lister) :

```
python mobile_scan.py --device --serial ZY12345678 --authorized
```

Sortie console : modèle, version Android, correctif de sécurité, verrou, chiffrement, SELinux, puis la liste des contrôles avec leur verdict (● critique, ● avertissement, ● ok) et la **note de posture /100**.

3. Rapport complet (JSON + HTML)

```
python mobile_scan.py --device --authorized -o /tmp/rapport_appareil
```

Produit `rapport_appareil.json` et `rapport_appareil.html` (autonome, lisible dans un navigateur) : informations de l'appareil, note de posture, chaque contrôle avec sa recommandation.

4. Les contrôles réalisés

Contrôle	Risque si mauvais	Sévérité
Verrou d'écran (aucun)	accès physique = accès aux données	 critique
Chiffrement du stockage (non chiffré)	données lisibles sans clé	 critique
adb réseau (adb tcpip)	écoute réseau sans câble	 critique
SELinux (Permissive)	isolation affaiblie	 critique
Fausse positions (mock location)	GPS falsifiable	 critique
Débogage USB activé	commandes depuis un PC (nécessaire pour l'audit)	
Bootloader déverrouillé / OEM unlock	firmware modifié possible	
Build userdebug/eng	accès root possible	
Services d'accessibilité actifs	lecture écran/frappes	
Sources inconnues / vérification off	sideload malveillant	
Vérification des applications (Play Protect) off	apps malveillantes	

La note part de 100 : **-25 par contrôle critique, -10 par avertissement** (bornée à 0).

5. Interpréter le résultat et durcir l'appareil

- **Score ≥ 80** : bonne posture. Pensez à **désactiver le débogage USB** après l'audit (c'est le seul avertissement habituel).
- **Score 40-79** : des points faibles réels (ex. pas de verrou, sources inconnues) — appliquez les recommandations de chaque contrôle.
- **Score < 40** : appareil très exposé (adb réseau, SELinux permissif, stockage non chiffré...) — durcissez avant usage quotidien.

Le rapport HTML détaille pour chaque contrôle la **recommandation**

exacte (paramètres à modifier, commandes fastboot, etc.).

6. Cas d'usage efficaces

1. **Avant de vendre / donner un appareil** : vérifiez qu'il n'y a pas d'accès résiduel (verrou, FRP, adb réseau) et faites une réinitialisation propre.
 1. **Parc d'entreprise** : auditez chaque appareil fourni aux employés et imposez un score minimum (ex. ≥ 80) avant déploiement.
 1. **Après une réparation** (écran changé, ROM réinstallée) : confirmez que le bootloader est re-verrouillé (état green) et SELinux Enforcing.
 1. **Appareil d'occasion acheté** : auditez-le avant d'y mettre vos données — un bootloader déverrouillé ou un adb réseau actif est un signal d'alerte.
-

7. Dépannage

Problème	Cause probable	Solution
« Aucun appareil connecté »	débogage USB désactivé ou invite non acceptée	Activez le débogage USB et acceptez l'invite sur l'appareil
« Plusieurs appareils connectés »	2+ appareils branchés	<code>adb devices</code> , puis <code>--serial <serial></code>
« adb introuvable »	outil absent	<code>sudo apt-get install -y adb</code>
Verrou « inconnu »	build exotique / ROM modifiée	Vérifiez manuellement : <code>adb shell dumpsys lock_settings</code>
Résultats identiques à chaque fois	cache adb	<code>adb kill-server && adb start-server</code>

8. Limites (honnêteté technique)

- L'audit lit l'état **actuel** de l'appareil : il ne détecte pas les malwares déjà installés, ni ne « déverrouille » quoi que ce soit.
- Le type de verrou est lu via `dumpsys` : sur certaines ROM (MIUI, OneUI...), la valeur peut être « inconnu » — le contrôle reste utile via les autres indicateurs.
- Il ne remplace pas une analyse de sécurité approfondie (le module `--android` complète pour les applications, le module `--wifi` pour le réseau).

Cadre d'utilisation : appareils que vous possédez ou êtes autorisé à tester. `--authorized` est obligatoire.

Généré par CodeScan — manuel imprimable.

IRON MAN AI — Contrôler votre téléphone

Manuel — Contrôler votre téléphone Android depuis votre PC

Voir l'écran de **votre** téléphone en temps réel et le piloter depuis votre ordinateur : à la maison (USB ou WiFi) ou à **distance** (depuis n'importe où dans le monde, via un tunnel sécurisé). La solution

principale est **scrcpy** (open-source, gratuit) + **adb**.

⚠ Cadre strict. Ce manuel ne s'applique qu'à **votre propre téléphone**

(ou un appareil dont vous avez la garde avec accord).

Le débogage USB exige l'accès à l'écran de l'appareil : c'est la

garantie que c'est le vôtre. Installer un accès distant sur le

téléphone de quelqu'un d'autre sans son consentement est un délit

(espionnage, accès non autorisé) — hors de propos ici.

1. Prérequis

- **adb** : `sudo apt-get install -y adb` (Linux) · `brew install android-platform-tools` (macOS) · [outils plateforme Google] (<https://developer.android.com/studio#command-line-tools>) (Windows).
- **scrcpy** : voir [§ 2 Installation](#2-installation).
- Votre téléphone en **débogage USB** : 1. Paramètres → À propos → tapez 7 fois sur « Numéro de build » ; 2. Paramètres → Options développeur → activez **Débogage USB** ; 3. Acceptez l'invite « Autoriser le débogage USB ? » sur l'écran (cochez « Toujours autoriser »).

2. Installation

Linux (Debian/Ubuntu)

```
bash scripts/install_scrcpy.sh          # installe scrcpy + adb + ffmpeg (sudo)
```

macOS

```
brew install scrcpy
brew install --cask android-platform-tools
```

Windows

Téléchargez le binaire sur

<<https://github.com/Genymobile/scrcpy/releases>> (fichier

`scrcpy-win64-vX.Y.zip`), décompressez-le, puis lancez `scrcpy.exe`.

Vérifiez :

```
adb devices      # votre téléphone doit apparaître comme « device »
scrcpy --version
```

3. Mode local — USB (le plus simple)

1. Branchez le téléphone en USB (débogage USB activé).

1. `adb devices` → votre téléphone apparaît en « device ».

1. Lancez :

```
scrcpy
```

L'écran du téléphone s'affiche dans une fenêtre. **Souris = toucher**,

clavier = saisie, glisser-déposer = transfert de fichiers, `Ctrl+C`

= copier, `Ctrl+V` = coller, `Ctrl+S` = capture d'écran, `Ctrl+0` =

écran éteint (mode économie), `Ctrl+X` = verrouiller l'écran.

4. Mode local — WiFi (même réseau)

Android 11 et plus (appairage sans fil)

1. Téléphone : Paramètres → Options développeur → **Appairage sans fil** (dans « Débogage sans fil »). Notez l'**adresse IP:port** et le **code d'appairage** affichés.

1. Sur le PC :

```
adb pair IP:PORT      # puis saisissez le code d'appairage
adb connect IP:PORT
scrcpy --tcpip
```

Android 10 et moins (adb tcpip)

```
adb devices                # téléphone en USB
adb tcpip 5555             # active adb réseau (1 fois)
# débranchez le câble, puis :
adb connect IP_DU_TELEPHONE:5555
scrcpy --tcpip
```

🔒 Après usage : `adb disconnect` et réactivez le débogage USB filaire

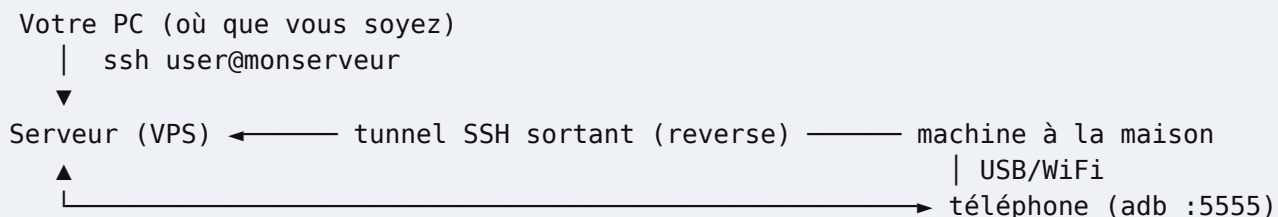
uniquement si nécessaire. Ne laissez **jamais** adb réseau actif sur un

réseau public (café, hôtel) : n'importe qui sur ce réseau pourrait

tenter de s'y connecter.

5. Mode distant — depuis n'importe où dans le monde

Principe : le téléphone reste **chez vous**, relié en USB (ou WiFi) à une machine allumée en permanence (PC ou mini-PC). Cette machine ouvre un **tunnel SSH sortant** vers un petit serveur que vous possédez (VPS à ~3 €/mois, ou un Raspberry Pi chez vous si vous avez une IP fixe/DNS dynamique). De n'importe où, vous vous connectez à votre serveur et pilotez le téléphone.



Étape 1 — machine à la maison (une seule fois)

```
# 1. Téléphone branché en USB, débogage USB activé
adb tcpip 5555
adb connect 127.0.0.1:5555          # adb local (sans le câble, en WiFi maison)

# 2. Clé SSH (une seule fois)
ssh-keygen -t ed25519
ssh-copy-id user@monserveur

# 3. Tunnel permanent (reverse) : expose le port adb local sur le serveur
ssh -N -R 5555:127.0.0.1:5555 user@monserveur
# → lancez-le via systemd/tmux pour qu'il survive aux déconnexions.
```

Étape 2 — depuis n'importe où

```
# Sur VOTRE PC portable, en voyage :
ssh user@monserveur -L 5555:127.0.0.1:5555  # terminal 1 (gardez ouvert)
scrcpy --tcpip=127.0.0.1:5555                # terminal 2 : pilotez le téléphone
```

Alternative : WireGuard (VPN privé)

Si vous avez un VPS, installez **WireGuard** (serveur sur le VPS, client sur votre PC) : le téléphone (via la machine à la maison) devient accessible comme un appareil du VPN, sans exposer de port. Plus simple à maintenir, chiffré de bout en bout.

Sécurité (non négociable)

- **Jamais** d'exposition directe d'adb sur Internet (pas de port 5555 ouvert publiquement) — toujours derrière un tunnel SSH ou un VPN.
- **Clés SSH uniquement** (`PasswordAuthentication no` sur le serveur), pas de mot de passe.
- Activez le **verrou d'écran** du téléphone et le chiffrement : le tunnel ne sert à rien si l'appareil est lui-même vulnérable.
- Désactivez le débogage USB / adb réseau quand vous n'en avez pas besoin.

6. Dépannage

Problème	Cause	Solution
adb devices vide	débogage USB off ou invite refusée	Activez le débogage USB, acceptez l'invite, adb kill-server
« device unauthorized »	clé de confiance non acceptée	Décochez « Toujours autoriser » et re-acceptez
adb connect échoue	IP erronée ou téléphone non appairable	Vérifiez l'IP dans les options développeur ; Android 11+ : appairage
scrcpy lent en WiFi	réseau saturé	Privilégiez le USB ; <code>scrcpy --max-size 1024 --max-fps 30</code>
écran noir	écran verrouillé ou vidéo non décodée	Déverrouillez le téléphone ; installez ffmpeg
tunnel distant coupé	déconnexion SSH	Utilisez <code>autossh</code> ou un service systemd avec redémarrage

7. Limites (honnêteté technique)

- scrcpy ne fonctionne que sur des appareils **avec débogage USB actif** (donc les vôtres). Il ne contourne aucun verrou.
- En mode distant, le téléphone doit rester allumé, chargé et relié à la machine à la maison (ou rester connecté en WiFi).
- La latence distante dépend de votre connexion ; le USB reste le plus fluide.
- Pour localiser ou verrouiller un téléphone **perdu**, utilisez **Google Find My Device** (officiel) — scrcpy ne s'y substitue pas.

Cadre : votre propre matériel. `adb` + `scrcpy` en local, tunnel SSH ou WireGuard pour le distant.